Large Language Models

How LLMs work, how they behave, and how that behaviour should shape the pipelines I build

Abhinav Ragidi

Started July 2026

Contents

What an LLM Is

> **What this chapter covers:** the mechanism in one page. Everything in the behaviour chapter later is a consequence of what's here, so it's worth being precise now.

Topic started July 2026. Parent note: `../ai-models.md`. *Provider-neutral throughout — this is about the class of model, not any one vendor's API.*

The mechanism, stated plainly

An LLM is a function that takes a sequence of tokens and returns a probability distribution over what the next token will be.

That's it. There is no second capability. Every impressive thing an LLM does — writing code, summarising a contract, holding a conversation, using a tool — is that one operation, applied repeatedly, with each new token appended to the input before the next call.

This is called **autoregressive generation**, and internalising it is the highest-leverage thing on this page, because a surprising number of practical behaviours fall out of it directly.

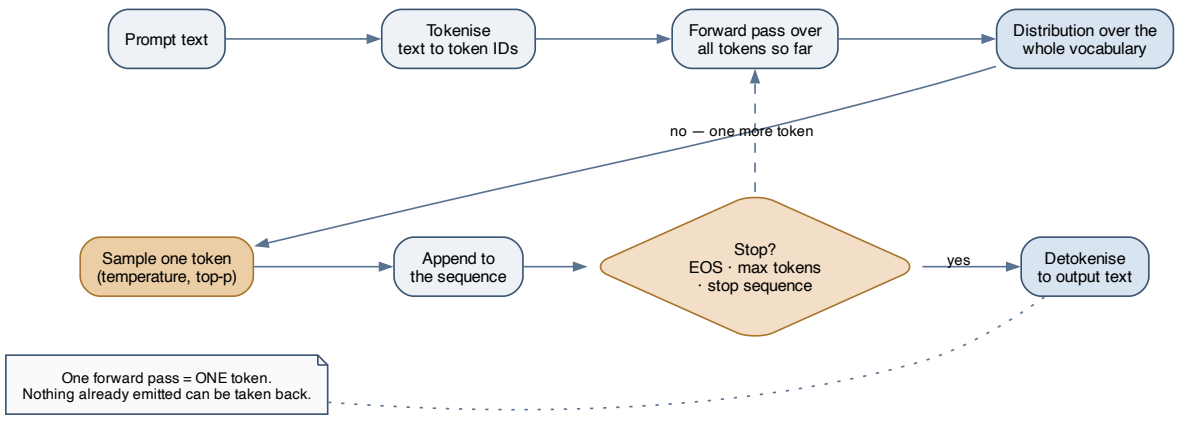

Figure 1: The generation loop. One forward pass produces exactly one token. The output grows by feeding the model its own output — and nothing already emitted can be revised.

Three immediate consequences:

- **Output cost scales with output length**, and each token requires a full pass over the network. Long answers are genuinely expensive, not just larger.

- **There is no backtracking.** Token 200 cannot change token 5. If the model commits to a wrong premise early, it will build fluently on top of it — the machinery has no mechanism for "actually, let me start over."
- **The "reasoning" you see is real computation.** Tokens spent working through a problem are forward passes that condition later tokens. This is why asking a model to think step by step measurably helps: you're giving it more compute, not just a different format.

Tokens, not words

Text is split into **tokens** — sub-word chunks. `unbelievable` might become `un` + `bel` + `ievable`. Common words are single tokens; rare words fragment.

Rough rule for English prose: **1 token 4 characters 0.75 words.** Code, JSON, non-Latin scripts, and unusual names all tokenise less efficiently, sometimes far less.

Why this matters beyond billing:

- **Every limit is in tokens** — context windows, output caps, rate limits, pricing. Never estimate them in characters or words, and never use another vendor's tokeniser to count for a different model family. Use the model's own counting endpoint or library.
- **Tokenisers differ between model families and even between versions**, so the same text can cost meaningfully different amounts on different models. Re-measure when you switch; don't apply a remembered multiplier.
- **Character-level tasks are unnaturally hard.** Counting letters in a word, reversing a string, strict rhyming — the model sees chunks, not characters. If you need that, do it in code.

The transformer block

Two alternating operations, repeated:

- **Self-attention — the mixing step.** Each token looks at every earlier token and decides how much each one matters to it. This is how "it" gets resolved to the right noun, and how an instruction at the top of the prompt influences a token near the bottom.
- **Feed-forward network — the thinking step.** Each token is transformed independently. This is where most parameters sit, and it acts like the model's recall and pattern-matching store.

The practical fact to carry forward: **attention compares every token against every other token, so cost grows quadratically with sequence length.** That single property explains why context windows are finite, why long prompts are slow as well as expensive, and why trimming context is one of the highest-leverage optimisations available.

The context window

The context window is **the total token budget for one request: prompt plus everything the model generates in reply.** System instructions, conversation history, retrieved documents, tool definitions, tool results, reasoning tokens, and the final answer all draw from the same pool.

Four things to hold about it:

1. **It is shared between input and output.** A tight output cap on a long prompt truncates the answer mid-sentence. If you enable a reasoning mode, reasoning tokens draw from the same budget — the answer needs headroom left over.

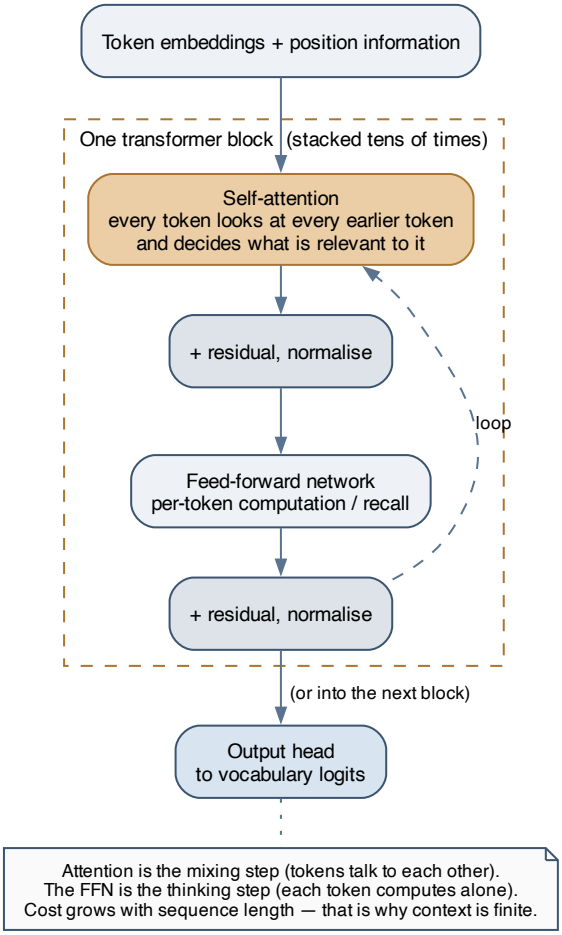

Figure 2: Inside a block: attention lets tokens exchange information, the feed-forward network does per-token computation. Stack this tens of times and you have the model.

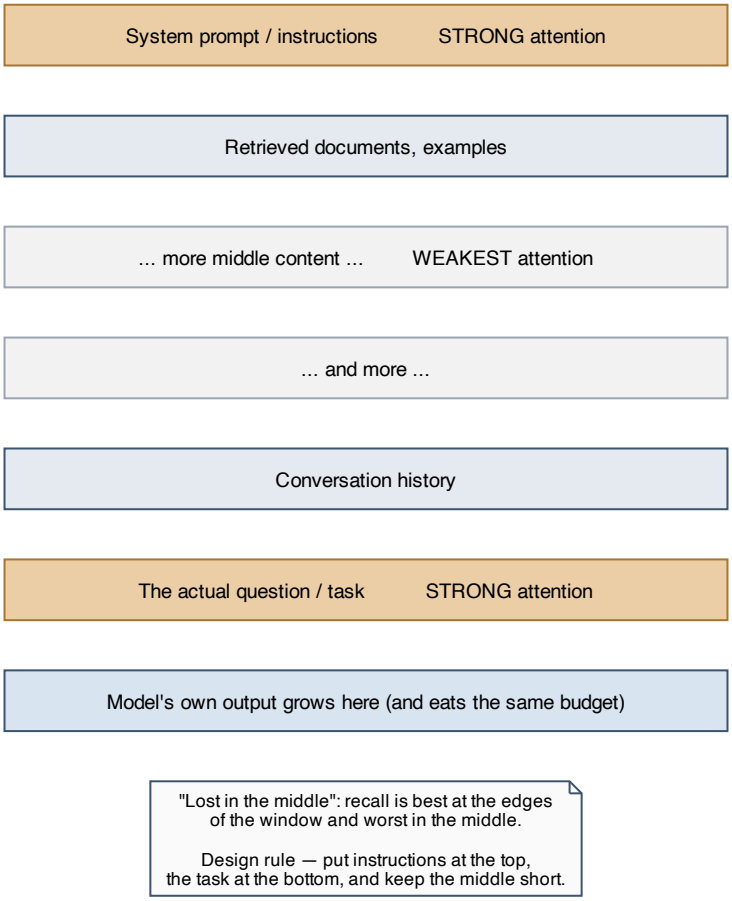

Figure 3: Recall across the window is uneven — strongest at the beginning and end, weakest in the middle. This is the "lost in the middle" effect, and it is a layout constraint you design around.

2. **A large window is not the same as good recall across it.** Models retrieve reliably from the edges and less reliably from the middle. "It fits" and "it will be used well" are different claims.
3. **Filling the window degrades quality, not just cost.** Irrelevant context is a distractor. Fewer, better-selected tokens routinely beat more tokens.
4. **The model is stateless between calls.** Multi-turn memory exists only because you re-send the history. Conversation state is *your* engineering problem, not the model's.

How It Was Trained

What this chapter covers: the training stages, because each one explains a different category of behaviour you'll observe at runtime.

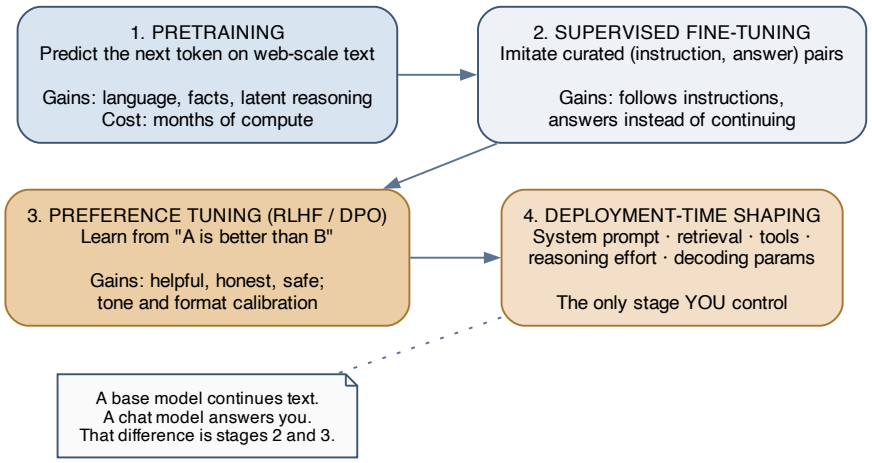

Figure 4: The stages. Pretraining creates capability, SFT creates instruction-following, preference tuning creates helpfulness and tone. Only the last stage is yours.

Stage 1 — Pretraining

Predict the next token, over an enormous corpus of text, for a very long time. No labels, no task — pure self-supervision.

What emerges: grammar, world knowledge, translation, arithmetic, code, and latent reasoning ability. None of it was taught explicitly; all of it turned out to be useful for predicting the next token.

The output is a **base model**, which is a *text continuation engine*, not an assistant. Ask it a question and it may well produce more questions — because that's what follows a question in a lot of training text.

This stage is where the **knowledge cutoff** and the **long-tail knowledge problem** come from. Facts seen a million times are solid; facts seen twice are unreliable — and the model has no way to tell you which situation it's in.

Stage 2 — Supervised fine-tuning

Continue training on curated `(instruction, good response)` pairs. This converts a continuation engine into something that answers.

Cheap relative to pretraining, and it's where the response *shape* is set: answer the question, use headings, refuse harmful requests.

Stage 3 — Preference tuning (RLHF / DPO)

Show humans two responses, record which they preferred, and train the model toward the preferred kind. This is where helpfulness, honesty, tone, and safety calibration are dialled in.

It also produces some quirks worth knowing:

- **Sycophancy.** Preferring what a human rated highly correlates with preferring what a human likes to hear. Push back on a correct answer and a model may cave. Never treat agreement as confirmation.
- **Formatting habits.** Bullet lists, hedged openings, restating the question. Artefacts of what raters rewarded, not of what your task needs.
- **Verbosity.** Longer answers often rated better, so length became a default. Concision usually has to be asked for explicitly.

Stage 4 — Deployment-time shaping (yours)

The only stage you control, and where all your leverage is:

- **The system prompt** — role, constraints, output contract.
- **Retrieval** — supplying facts the weights don't reliably contain.
- **Tools** — letting the model act and observe rather than guess.
- **Reasoning depth** — many current models expose a knob for how much thinking to spend; higher settings buy accuracy on hard tasks and cost latency and tokens on easy ones.
- **Decoding parameters** — see below.

Note that on newer model generations, some classic knobs are disappearing in favour of higher-level controls: models increasingly decide their own thinking depth from an effort hint, and some no longer accept sampling parameters at all. **Check the current contract of the specific model rather than assuming the parameter set you learned earlier still applies.**

Decoding — turning a distribution into text

Each step yields a distribution over the vocabulary. How you pick from it:

Control	What it does	How I set it
Temperature	Flattens or sharpens the distribution. Low = predictable, high = varied	Low for extraction, classification, code. Higher only when variety is the goal
Top-p (nucleus)	Sample only from the smallest set of tokens covering probability mass p	Adjust this *or* temperature, not both — they interact confusingly

Control	What it does	How I set it
Top-k	Sample only from the k most likely tokens	Rarely my first lever
Max output tokens	Hard cap on generation	Generous enough to finish. Truncation costs a whole retry
Stop sequences	Halt when a string appears	Useful for delimited formats
Seed	Where offered, reduces run-to-run variation	Helps reproduce a bug; not a determinism guarantee
Structured output / schema	Constrains output to a JSON schema	**The single highest-value control for pipelines.** Prefer it over "please return JSON"

Temperature 0 is not determinism. It makes the *sampling step* greedy, but batching, hardware, and routing still introduce variation. Design for "usually the same", never "provably the same".

Model Behaviour and Pipeline Design

What this chapter covers: the chapter I actually want. Each section names an observable behaviour, explains the mechanism that causes it, and states the design rule it implies. The behaviour is not a bug to be prompted away — it's a property to build around.

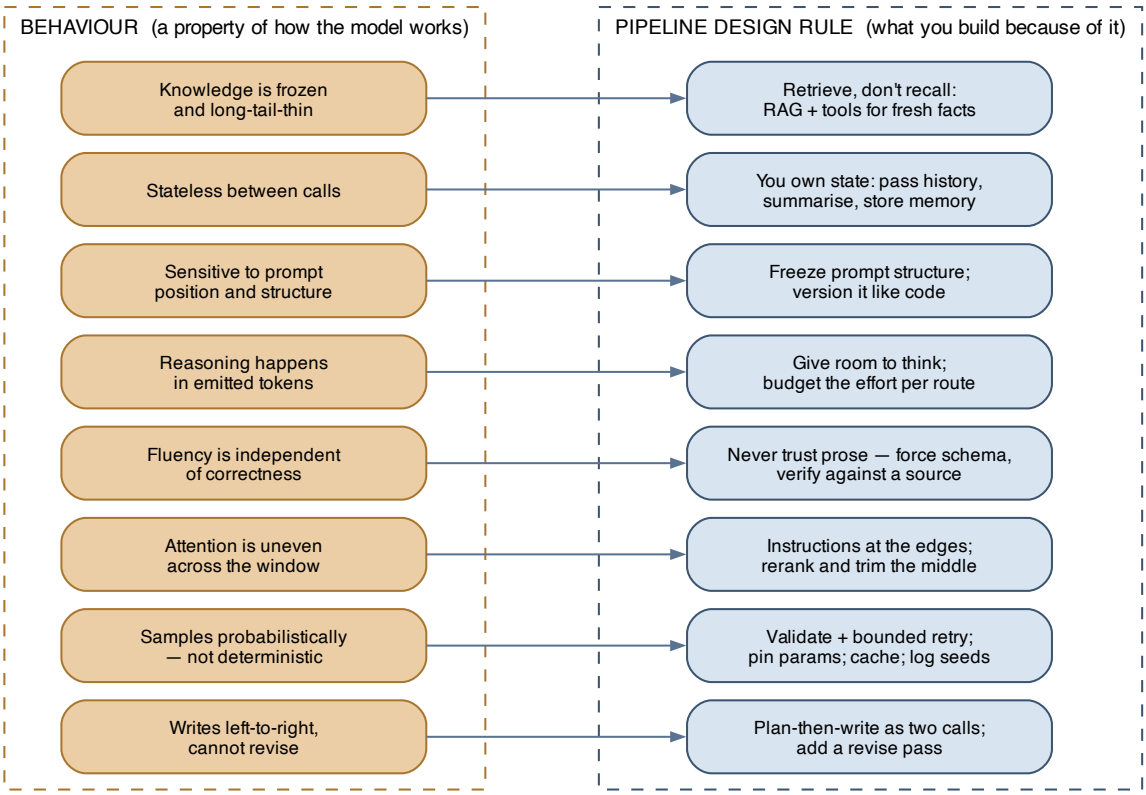

Figure 5: The mapping in one view: each behaviour on the left forces a corresponding structural choice on the right. Design the pipeline from the left column, not from what you wish the model did.

The governing principle for everything below:

> **Design so the model does what it is reliably good at, and put everything it is unreliable at into code around it.**

Models are reliably good at language transformation, pattern recognition, extraction from provided text, drafting, and classification with clear criteria. They are unreliable at exact arithmetic, precise recall of long-tail facts, strict format compliance without enforcement, knowing what they don't know, and anything requiring a guarantee. Draw the boundary there.

B1 — It writes forward and cannot revise

Behaviour. Output is committed token by token. A wrong early assumption gets built on fluently for the rest of the answer. Ask for a document with a conclusion at the top and the conclusion is written *before* the analysis that supposedly supports it.

Mechanism. Autoregression. There is no edit pass.

Design rules.

- **Split planning from producing.** One call to outline or decide the approach, a second call to write against that plan. The plan then conditions everything downstream instead of being invented mid-flight.
- **Put conclusions last, or add a second pass.** If the format demands a summary up front, generate the body first, then generate the summary from it.
- **Add a revise stage for anything high-stakes.** A separate call that sees the output and critiques or corrects it is doing what the model cannot do internally. A *fresh* context critiquing the output usually beats asking the same call to self-check.
- **Let it think before it commits.** Reasoning tokens, or an explicit "work through it before answering" instruction, give it room to be wrong cheaply before the answer starts. Some newer models over-verify when instructed to double-check, so measure rather than adding self-check instructions reflexively.

B2 — It is probabilistic, not deterministic

Behaviour. The same input can produce different output. A prompt that worked ten times fails the eleventh.

Mechanism. Sampling, plus non-determinism in the serving stack.

Design rules.

- **Never let unvalidated output reach a consumer that assumes structure.** Parse, validate against a schema, and check business invariants. Treat the model as an untrusted input source — because that is exactly what it is.
- **Retry with bounds, and feed the error back.** Include the validation failure in the retry so the second attempt has information the first didn't. Cap retries and define what happens when they're exhausted.
- **Test on distributions, not examples.** One success proves nothing. Build a set of cases and measure a pass rate — that number is your only real signal that a prompt change helped.
- **Log inputs and outputs.** Without the exact prompt that produced a bad output, you cannot reproduce or fix it.

B3 — Attention across the context is uneven

Behaviour. A constraint buried in the middle of a long prompt gets ignored. Retrieval quality drops as you stuff in more documents, even below the window limit.

Mechanism. Position effects in attention plus dilution — more content competing for the same attention.

Design rules.

- **Layout is a design decision.** Stable instructions at the top, the specific task and question at the bottom, bulk reference material in the middle where weak recall hurts least.
- **Rerank and trim, don't dump.** Retrieving 30 chunks and passing all of them is worse than retrieving 30, reranking, and passing the best 5. Precision beats recall once it's in the prompt.
- **Repeat critical constraints at the end.** A short restatement of the non-negotiable rule immediately before the task is cheap and effective.
- **Summarise long history rather than carrying it verbatim.** Rolling summaries keep the salient state and drop the noise.
- **Treat "it fits in the window" as necessary, not sufficient.** Measure recall on your actual long inputs.

B4 — Fluency is independent of correctness

Behaviour. Hallucination: confident, well-formed, plausible, wrong. Invented citations, invented API methods, invented numbers. No hedging, no signal.

Mechanism. The objective rewards plausible continuations. A fabricated-but-plausible citation scores well against "predict the next token"; there is no term in the loss for *true*.

This is the most important behaviour in the chapter, because it inverts the normal relationship between a system's confidence and its reliability.

Design rules.

- **Ground every factual claim in supplied text.** Retrieval isn't primarily about freshness — it's about giving the model something to be correct *from*, instead of recalling from weights.
- **Require citations, then verify them mechanically.** Ask for the source span, and check in code that the quoted span actually appears in the source document. A model that must cite fabricates less; a pipeline that verifies citations catches what's left.
- **Never let the model do arithmetic or exact lookups that a tool can do.** Give it a calculator, a query interface, a code interpreter. Route computation to something that computes.
- **Constrain the answer space where you can.** "Pick one of these enum values" or "return `null` if not present" is far more reliable than open generation — and gives explicit permission to decline, which reduces invention.
- **Calibrate review to blast radius.** Reversible, low-stakes, cheap to check → let it run. Irreversible or expensive to undo → human in the loop, always. The cost of a wrong answer is the correct input to that decision, not the model's apparent confidence.

B5 — Reasoning is computation done in tokens

Behaviour. Forcing an immediate answer on a multi-step problem produces worse answers than letting it work through the steps. Asking for "just the number" on a hard question is asking it to guess.

Mechanism. Each generated token is a forward pass that conditions the next. Intermediate tokens are literally where multi-step computation happens.

Design rules.

- **Never demand a bare answer to a hard question.** Let it reason, then extract the final answer — in a second call, or as a designated field in a structured response.
- **Budget reasoning per route, don't set it globally.** Deep reasoning on a classification task is wasted latency; shallow reasoning on a hard analysis is wrong answers. Tier your routes.
- **Decompose instead of escalating.** Three focused calls often beat one call with maximum reasoning, and each step is independently testable and debuggable. Prefer decomposition before reaching for the biggest model.
- **Give room in the token budget.** Reasoning consumes output tokens. A cap sized for the answer alone gets you reasoning followed by a truncated answer.

B6 — It is sensitive to prompt structure

Behaviour. Reordering sections, changing a heading, or rewording an instruction changes behaviour, sometimes materially. Emphatic instructions ("CRITICAL: you MUST always…") can cause *over*-triggering on capable models — reaching for a tool constantly, or applying a rule where it doesn't fit.

Mechanism. The prompt is the input. All of it conditions the output, including its structure. And instruction-following has become strong enough that emphatic phrasing is now taken literally.

Design rules.

- **Treat prompts as versioned artefacts.** In source control, with a changelog, and with eval results attached to changes. A prompt is production configuration.
- **Change one thing at a time and measure.** Otherwise you cannot attribute an improvement or a regression.
- **Structure explicitly.** Clear sections with headings or tags: role, constraints, input, output contract. Structure makes instructions findable and keeps the stable prefix stable.
- **State conditions rather than shouting.** "Use the search tool when the answer depends on current information" beats "ALWAYS SEARCH FIRST". Turn emphasis into a trigger condition.
- **Re-tune on model change.** A prompt tuned against one model — especially one full of workarounds for that model's quirks — can *underperform* on a newer one, because the workarounds now fight behaviour that no longer needs correcting. Migration means re-running evals, not just swapping an identifier.

B7 — It is stateless

Behaviour. No memory between calls. Everything it appears to remember was re-sent.

Mechanism. Frozen weights, single forward pass per request.

Design rules.

- **Own the conversation state explicitly.** Store it, decide what to include, decide what to drop.
- **Have a strategy for history growth before you need one.** Rolling summarisation, dropping stale tool output, or a compaction step — chosen deliberately rather than discovered when you hit the limit in production.

- **Persist anything durable outside the model.** A database, a file, a memory store. Long-running agents work far better with somewhere to write notes for their future selves.
- **Keep the stable prefix byte-identical to exploit caching.** See the cost chapter — this is where statelessness pays you back.

B8 — Knowledge is frozen and thin in the long tail

Behaviour. Wrong on recent events. Wrong on your internal systems. Wrong on obscure specifics — often *confidently*, and often by describing how something plausibly would work.

Mechanism. Weights were fixed at training time; rare facts got little signal.

Design rules.

- **Retrieve, don't recall.** For anything time-sensitive, proprietary, or niche, put the fact in the prompt.
- **Assume it does not know your domain.** Internal service names, schemas, and conventions must be supplied. A plausible-sounding description of your own system is the classic failure here.
- **Prefer tools over training for freshness.** Fine-tuning is a poor mechanism for facts — expensive, immediately stale, and it doesn't reliably override pretrained knowledge anyway.

The failure-mode table

The version of this I actually want to reach for when something is wrong:

Symptom	Likely cause	First fix
Confidently wrong facts	Recalling from weights (B4, B8)	Retrieve and require verifiable citations
Output won't parse	Unconstrained generation (B2)	Schema-constrained output + validate + bounded retry
Ignores an instruction	Buried in the middle (B3)	Move it to the edges; restate before the task
Answer cut off mid-sentence	Output cap too low, or reasoning ate the budget (B1, B5)	Raise the cap; account for reasoning tokens
Bad on multi-step problems	Forced to answer immediately (B5)	Allow reasoning; or decompose into steps
Inconsistent across runs	Sampling (B2)	Lower temperature; constrain output; validate
Arithmetic errors	Doing maths in tokens (B4)	Hand it a tool
Agrees with a wrong correction	Sycophancy (stage 3)	Ask for evidence; don't treat agreement as confirmation
Too verbose	Length preference from tuning	Ask for concision explicitly; cap tokens; constrain format
Over-uses a tool	Emphatic instruction over-triggering (B6)	Soften to a trigger condition
Quality drops as context grows	Dilution (B3)	Rerank and trim; summarise history

Symptom	Likely cause	First fix
Regressed after a model upgrade	Prompt tuned to the old model (B6)	Re-run evals; strip stale workarounds

Pipeline Patterns

What this chapter covers: the compositions I keep reaching for, and the signal that tells me to move up a level of complexity.

Pattern	Use when	Cost
Single call	The task fits one step and errors are cheap	Lowest
Decompose	One call is unreliable but each sub-step is easy	Low — and each step is testable
Retrieve then generate (RAG)	Answers depend on facts outside the weights	Medium; retrieval quality dominates the outcome
Validate + bounded retry	Output feeds code that assumes structure	Low overhead, largest reliability gain per unit effort
Tool / agent loop	The model must act on the world, or observe to decide	High and variable — bound it
Judge / ensemble	Quality is subjective, or stakes justify the spend	Highest — N generations plus scoring

The default I want to hold: **single call, wrapped in validate-and-retry.** That combination handles a surprising share of real work. Add retrieval when facts are the problem, decomposition when reliability is, and an agent loop only when the task genuinely requires acting rather than answering.

Notes on the two expensive ones:

- **Agent loops must be bounded on every axis** — max steps, max tokens, max wall-clock, max spend. An unbounded loop is a runaway cost incident waiting to happen. Gate irreversible actions behind explicit approval, and remember that a tool result entering the context is untrusted input (B4 applies to tool output too).
- **Judge patterns work better with diverse angles than repeated identical attempts.** Three verifiers each looking through a different lens catch failure modes that three copies of the same verifier all miss together.

Choosing an adaptation strategy

The recurring "should I fine-tune?" question, resolved by asking what's actually broken:

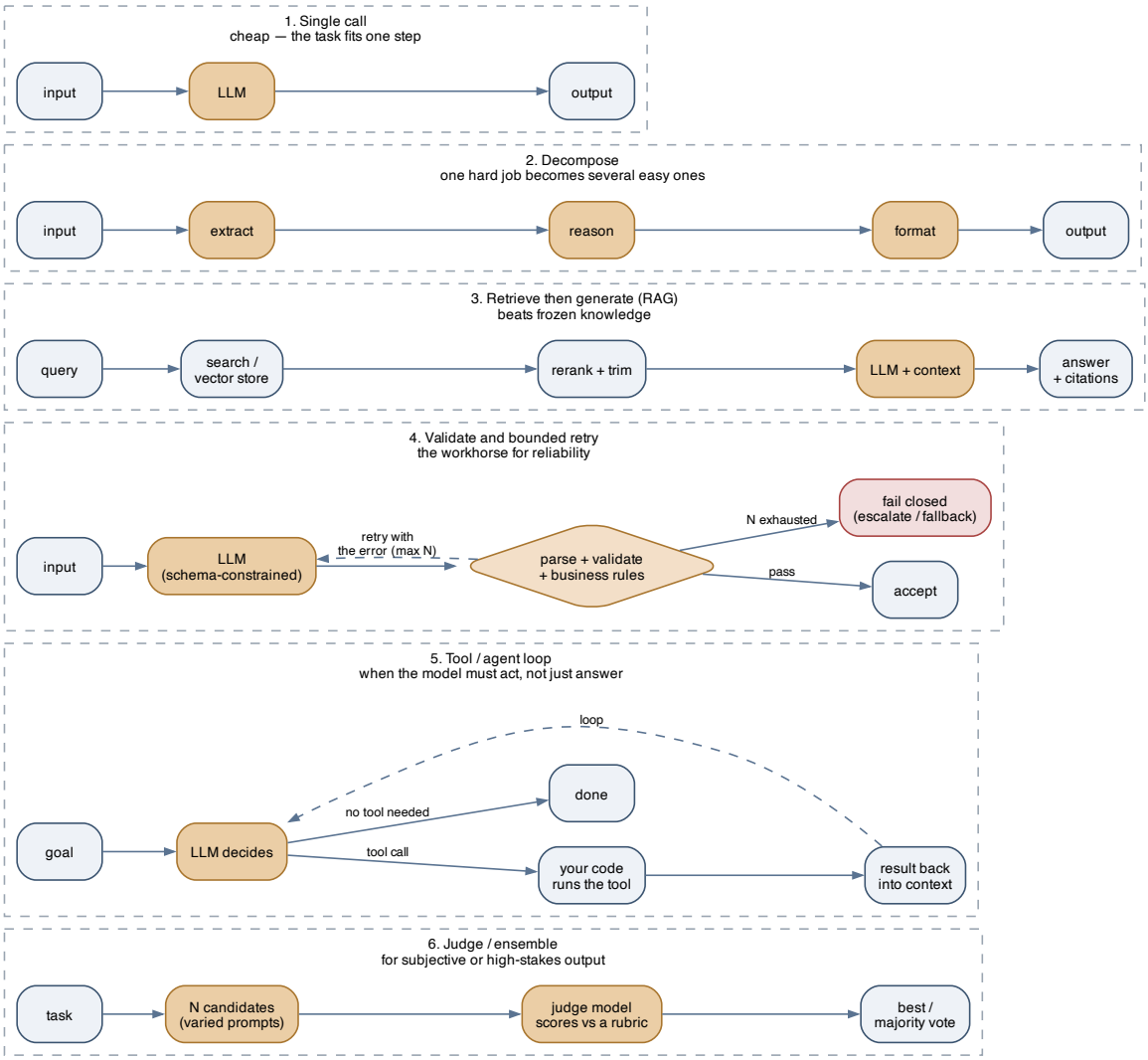

Figure 6: Six patterns, roughly in order of increasing cost and capability. Start at the top and move down only when a measured failure forces it.

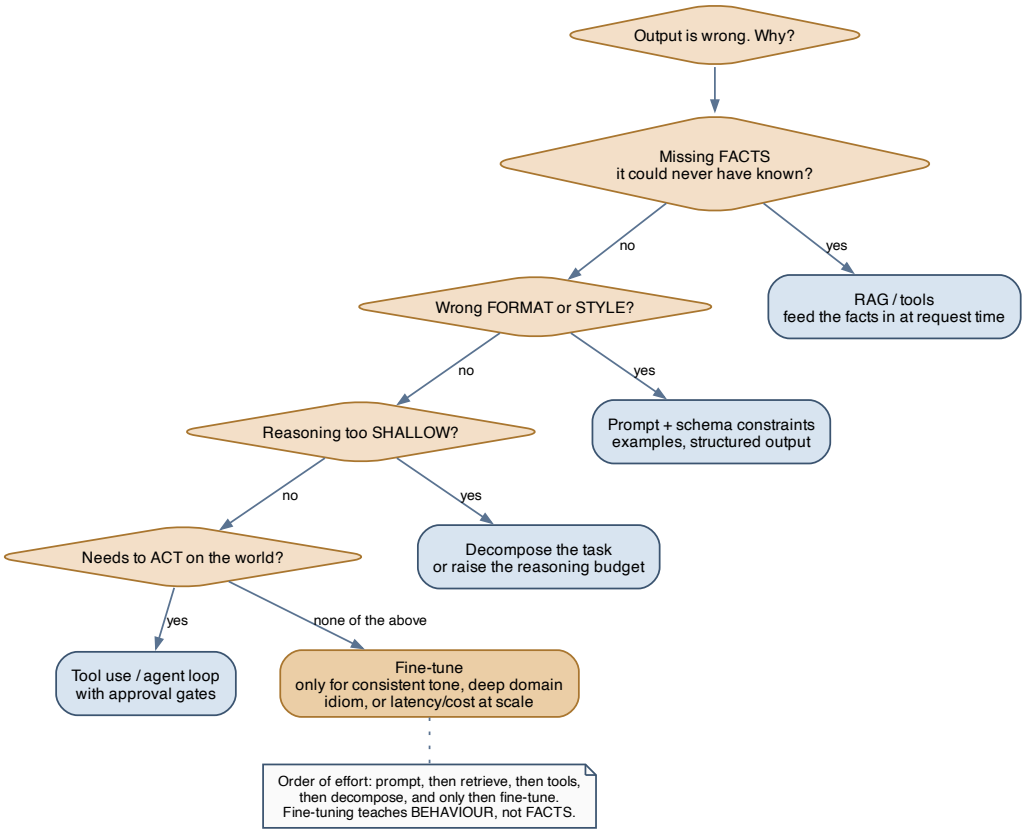

Figure 7: Diagnose the failure first. Fine-tuning is far down the list, and it teaches behaviour rather than facts.

Approach	Teaches	Reach for it when
Prompting	Nothing — steers existing ability	Always first. Fastest to iterate
Structured output	Nothing — constrains the shape	Whenever output feeds code
RAG	Facts, at request time	Knowledge is missing, private, or changing
Tools	Capability and current state	Exact computation, or acting on the world
Decomposition	Nothing — reduces per-step difficulty	Single-call reliability is too low
Fine-tuning	Behaviour, tone, format, domain idiom	Consistent style at scale, or cost/latency at volume — after the cheaper options are exhausted

The rule worth memorising: fine-tuning teaches behaviour, retrieval supplies facts. Fine-tuning to inject knowledge is the most common expensive mistake in this space.

Evaluation and Operations

What this chapter covers: the part that makes an LLM feature maintainable rather than a demo.

Evals are the substitute for a type system

You cannot unit-test a probabilistic function on one input. What replaces it is a **dataset of cases with a pass criterion, and a tracked pass rate.**

- **Start small and grow from failures.** Twenty real cases beat a thousand synthetic ones. Every production bug becomes a permanent case — this is how the suite gets genuinely valuable over time.
- **Match the grader to the task.** Exact match or schema validation for extraction. Deterministic assertions where possible. An LLM judge with an explicit rubric for open-ended output — and validate the judge against human labels before trusting it, because a miscalibrated judge is worse than no judge.
- **Gate changes on the pass rate.** Prompt edits, model upgrades, and retrieval changes all need a before-and-after number. Without it you're guessing.

What to log and watch

Layer	Track
Request	Full prompt, model identifier, parameters, output, latency, token counts
Quality	Validation failure rate, retry rate, refusal rate, eval pass rate over time
Cost	Tokens per request, cache hit rate, spend per feature, calls per user task
Loops	Steps per task, tool error rate, how often bounds are hit
Human	Escalation rate, correction rate, user thumbs-down

The rate to watch hardest is silent failures — output that parsed and shipped but was wrong. It won't appear in error logs, so it needs sampling, review, and a user feedback path.

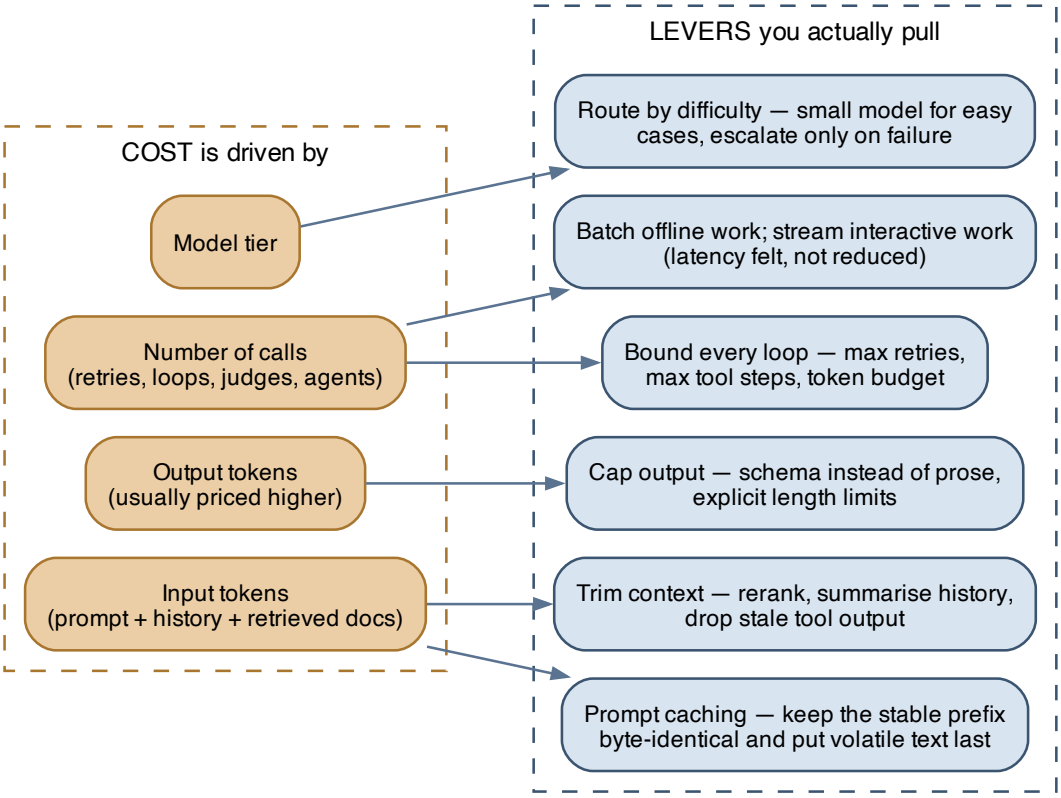

Figure 8: Cost is tokens times calls times tier. Each driver has a specific lever.

Cost and latency

The levers in the order I'd apply them:

1. **Prompt caching.** Providers commonly cache a stable prompt *prefix*, and matching is exact from the start of the request. So keep the prefix byte-identical: no timestamps, session IDs, or randomly-ordered serialisation early in the prompt, and put volatile content last. A single interpolated timestamp near the top can silently disable caching for the whole request — this is the most common self-inflicted cost bug in the space, and it's invisible unless you're watching cache-hit metrics.
2. **Trim context.** Rerank retrieved chunks, summarise history, drop stale tool output. Cheaper *and* usually better output (B3).
3. **Cap output.** Structured output instead of prose, explicit length limits. Output tokens are typically the more expensive direction.
4. **Route by difficulty.** Small fast model for the easy majority, escalate on failure or on a difficulty signal. Often the single largest saving available.
5. **Bound every loop.** Retries, tool steps, agent iterations, total token budget.
6. **Batch what's offline; stream what's interactive.** Streaming doesn't reduce latency, it relocates the perception of it — which for a user-facing feature is most of the battle.

On latency specifically: **time to first token** and **tokens per second** are separate problems. Long prompts hurt the first; long outputs hurt the second. Trimming context fixes one, capping output fixes the other, and confusing them wastes optimisation effort.

Key Takeaways

One-line summary. An LLM is an autoregressive next-token predictor whose useful behaviours and characteristic failures both fall out of that mechanism — so the right way to design a pipeline is to read off each behaviour (no revision, probabilistic, uneven attention, fluent-but-not-correct, reasoning-in-tokens, structure-sensitive, stateless, frozen knowledge) and put code around the model wherever it needs a guarantee it cannot give.

The five rules I want to carry into every pipeline:

1. **Validate everything.** The model is an untrusted input source. Schema-constrain, parse, check invariants, retry with bounds, and define the fail-closed path.
2. **Retrieve facts, don't recall them.** Ground factual claims in supplied text with verifiable citations. Fine-tuning is for behaviour, not knowledge.
3. **Decompose before escalating.** Several small reliable steps beat one large fragile one, and each is testable.
4. **Layout and budget are design decisions.** Instructions at the edges, middle kept short, reasoning given room, output capped generously, loops bounded.
5. **Evals are the contract.** No prompt change, model upgrade, or retrieval tweak ships without a before-and-after pass rate.

Questions to revisit:

- How much does reranking retrieved chunks actually move end-to-end answer quality on my own data, versus just retrieving fewer?
- Where's the crossover point at which routing to a bigger model beats decomposing into smaller calls — on cost, and on quality?
- How do I detect silent failures at a useful rate without reviewing everything by hand?
- How reliable is an LLM judge on my task, measured against human labels, before I let it gate anything?
- What does re-tuning a mature prompt for a new model generation actually involve in practice — how much of it is stripping old workarounds?